复旦大学 计算机系统基础

Lab2 BombLab 实验报告

x86-64 汇编分析与动态口令恢复

表 1　实验信息

项目	内容
学生姓名	叶梓铭
学号	24301030033
学期	2026 秋季学期
实验环境	Windows 11 / WSL2 / Ubuntu 22.04 / VS Code WSL
个人仓库	https://github.com/yzm7678024-glitch/ics-2026-bomblab

2026 年 10 月 30 日

1 实验目的

本次 BombLab 实验需要对个人领取的二进制炸弹进行逆向分析，理解六个正式关卡及 Honor 的口令校验逻辑，并在 defuser.c 中实现能够根据不同 challenge 动态生成正确口令的程序。与寻找固定密码的传统 BombLab 不同，本实验不仅要求通过测试，更重要的是从汇编代码中还原程序的计算过程，理解其背后的实现原理。

实验过程中，主要通过反汇编和 GDB 调试，分析函数调用、循环、跳转表、递归、链表及函数指针等结构在 x86-64 汇编中的具体实现。Honor 部分还涉及字符串的内存布局、小端序以及 context 的传递。通过将汇编分析结果转化为 C 语言代码，并使用不同 seed 进行实际验证，进一步加深对程序控制流、数据流和内存访问方式的理解，提高分析和解决实际程序问题的能力。

2 实验环境与分析方法

2.1 实验环境与资料

表 2 实验环境与材料用途

项目	本次使用方式
运行环境	Windows 11 主机，WSL2 中的 Ubuntu 22.04，VS Code WSL；截图连接名称为 Ubuntu-22.04。
目标与工具	Linux x86-64 ELF；GCC、GNU Make、Python、GDB、objdump。编译与运行在 WSL 中进行。
个人解法	已完成的个人仓库 defuser.c；接口和公开测试脚本用于核对调用及验证方式。
原始证据	个人 bomb 反汇编文件、各关公开测试截图，以及额外 seed 完整验证截图。

2.2 分析方法与位宽约定

本实验主要结合 objdump 反汇编和 GDB 调试分析各关的校验逻辑。分析时，首先确定输入格式及最终的比较条件，再根据寄存器中的数据变化，逐步追踪期望值的计算过程。对于涉及函数调用、间接跳转或指针访问的部分，还需要进一步分析相关辅助函数和内存数据，最终将汇编逻辑还原为相应的 C 语言计算关系。下文主要列出各关涉及的关键汇编指令，省略常规的函数序言等内容。汇编采用 AT&T 语法，源操作数在前，目的操作数在后。

下文令 C 表示当前关卡的 64 位无符号 challenge，K 表示 Honor 的 64 位 context。U32(z) 表示保留 z 的低 32 位；$\oplus$ 表示逐位异或，& 表示按位与，>> 表示逻辑右移。除特别说明外，恢复式中的加、减、乘及左移均按 uint32_t 保留低 32 位。右移 challenge 时先在 64 位域完成，再截取结果。

$$U32(z) = z \bmod 2^{32}; \quad H = U32(C >> 32); \quad L = U32(C)$$

整数与指针参数通常通过 rdi、rsi、rdx、rcx、r8、r9 传递，32 位返回值放在 eax 中。但寄存器会被重复利用，分析时必须以指令执行到的位置为准。例如 Phase 4 的 eax 在递归调用前后就表示不同的值。跳转表和节点数据则以个人炸弹中的内存布局为依据。[1]–[4]

3 Phase 1 函数调用与数据流

3.1 关键汇编与参数流

phase_01 先调用 valid_u32_fields，要求恰好输入一个无符号整数；末尾再用 p1_compute 计算期望值，与输入比较。最关键的是 p1_compute 中三次辅助函数调用及其参数传递。

```
# phase_01
40209d: mov $0x1,%esi
4020aa: call 402c50 <valid_u32_fields>
4020d3: cmp $0x1,%eax
4020db: call 402050 <p1_compute>
4020e0: cmp %eax,-0x14(%rbp)

# p1_step0 / p1_step1 / p1_step2
402024: lea 0xca5e(%rsi),%eax
40202a: xor %edi,%eax
402034: lea (%rdi,%rsi,8),%eax
402037: xor $0x1604,%eax
402044: sub %esi,%edi
402046: lea 0x92a9(%rdi,%rdi,4),%eax

# p1_compute
40205e: shr $0x20,%rax
402062: shr $0x10,%rdx
40206e: call 402020 <p1_step0>
40207b: call 402030 <p1_step1>
402085: call 402040 <p1_step2>
40208b: xor %ecx,%eax
```

p1_compute 用到了 C 的低 32 位 L、高 32 位 H，以及右移 16 位后截取的 M。这里 M 并不是独立的 16 位字段，它覆盖了 C 高、低两半的一部分。三个辅助函数接收的参数各不相同：p1_step0 使用 L、H，p1_step1 使用 H、M，p1_step2 则使用前一步算出的 b 和 L。

$$M = U32(C >> 16);\quad a = (H + 0xca5e) \oplus L$$

$$b = (H + 8M) \oplus 0x1604;\quad c = 5(b - L) + 0x92a9;\quad y = c \oplus a$$

这里的 lea 主要用来完成加法和常数倍运算，不涉及内存读取。比如 0xca5e 就是十进制 51806。第一次辅助函数得到的 a 暂存在 ecx 中，最后与第三次调用的返回值异或，得到这一关需要输入的整数。

```
a=(h+51806)^l;
b=(h+m*8)^0x1604;
c=(b-l)*5+0x92a9;
values[0]=c^a;
```

3.2 运行验证

图 1 保留了本关 10/10 的公开测试结果，以及 seed 20261030 下的单关验证。该次 challenge 为 0x30348194e1f8df4d，炸弹显示“Phase 1 defused.”。

```
yeziming@yzm:~/ICS2026/ics-2026-bomblab$ python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
Phase 1 challenge: 0x30348194e1f8df4d
Input (unsigned decimal integers):
Phase 1 defused.
```

图 1　Phase 1 公开测试与额外 seed 验证

4 Phase 2 循环与递推

4.1 首项与循环体的区别

phase_02 要求六个整数。第 0 项单独计算，后五项则通过循环逐项比较。循环中 ecx 从 10 开始，每轮增加 5，决定从 challenge 中提取哪六位；edx 从 1 递增到 5，对应当前项的编号。

```
# phase_02 首项
4020fd: mov $0x6,%esi
402160: shr $0x1e,%rax
402164: and $0xf,%eax
402167: xor $0x1b,%eax
40216a: cmp %eax,%esi
# 递推循环
40216e: mov $0xa,%ecx
402173: mov $0x1,%edx
402178: cmp $0x6,%rdx
40217c: je 402117 <phase_02+0x27>
40217e: mov %r13,%rax
402181: shr %cl,%rax
402184: add $0x5,%ecx
402187: and $0x3f,%eax
40218a: xor %edx,%eax
40218c: lea (%rax,%rsi,4),%eax
40218f: mov -0x40(%rbp,%rdx,4),%esi
402193: add $0x1,%rdx
402197: cmp %eax,%esi
402199: je 402178 <phase_02+0x88>
```

$$v_0 = ((C >> 30)\,\&\,15) \oplus 27$$

$$a_i = ((C >> (5 + 5i))\,\&\,63) \oplus i; \quad v_i = U32(4v_{i-1} + a_i), \qquad 1 \le i \le 5$$

shr %cl,%rax 中的移位量依次为 10、15、20、25、30；and $0x3f,%eax 提取六位；xor %edx,%eax 再加入迭代编号。lea (%rax,%rsi,4),%eax 说明当前期望值还要加上前一项的 4 倍，所以后五项必须依次计算，不能分别求值。

实现先在 64 位 challenge 上移位，再将结果赋给 uint32_t a。随后的掩码和运算均为 32 位。通用恢复应保留回绕语义；本关首项位于 16～31，每次附加项至多 63，五轮后上界为 $31\times4^5+63\times(4^5-1)/3=53227$，因而合法路径实际上不会发生 32 位溢出。

```
values[0]=((challenge>>30)&15)^27;
for(int i=1;i<=5;i++) {
    a=challenge>>(5+i*5);
    a=(a&0x3f)^i;
    values[i]=a+values[i-1]*4;
}
```

4.2 运行验证

图 2 显示本关公开测试 10/10。seed 20261030 对应的 challenge 为 0x0499d9767b99a822，输入生成的六项后显示“Phase 2 defused.”。

```
python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
Phase 2 challenge: 0x0499d9767b99a822
Input (unsigned decimal integers):
Phase 2 defused.
```

图 2 Phase 2 公开测试与额外 seed 验证

5 Phase 3 跳转表与 switch

5.1 selector 与间接跳转

phase_03 输入两个整数：第一个必须等于 C 的最低三位；第二个与 p3_dispatch 的返回值比较。右移 3 位和 32 位分别在 64 位寄存器中完成，再以 eax、esi 异或，因此计算参数为下式中的 x。

$$s = C\ \&\ 7;\quad x = U32(C >> 32) \oplus U32(C >> 3);\quad 输出 = (s, D_s(x))$$

```
# phase_03 提取与比较
4022f1: and $0x7,%edi
4022f4: cmp %edi,-0x18(%rbp)
4022ff: shr $0x3,%rax
402303: shr $0x20,%rsi
402307: xor %eax,%esi
402309: call 402240 <p3_dispatch>
40230e: cmp %eax,-0x14(%rbp)
```

```
# p3_dispatch
402244: mov %edi,%eax
402246: mov %esi,%edi
402248: cmp $0x7,%eax
40224b: ja 402292 <p3_dispatch+0x52>
40224d: notrack jmp *0x405820(,%rax,8)
402268: jmp 4021b0 <p3_case_1>
402270: jmp 4021a0 <p3_case_0>
402278: jmp 4021f0 <p3_case_4>
402280: jmp 4021c0 <p3_case_2>
402288: jmp 402220 <p3_case_6>
402290: jmp 402230 <p3_case_7>
402258: jmp 4021d0 <p3_case_3>
402260: jmp 402200 <p3_case_5>
```

p3_dispatch 把 selector 放入 eax，把运算参数 x 移到 edi。notrack jmp *0x405820(,%rax,8)会从地址 0x405820+8×selector 处读取跳转目标，再转到对应的代码块。这里每个表项都是一个 8 字节的地址，不能把表项的位置和分支函数的编号混为一谈。

这张表并不是按 p3_case_0、p3_case_1 依次排列。例如 selector=0 跳到的是 p3_case_1，而不是 p3_case_0。我将八个索引分别对应的分支整理到表 3，再逐个核对分支中的实际运算。

表 3　Phase 3 索引、函数与 32 位运算

s	目标函数	$D_s(x)$	关键指令地址
0	p3_case_1	$(2x) \oplus 0xacac$	4021b4、4021b7
1	p3_case_0	$3x + 0xd61d$	4021a4
2	p3_case_4	$5x + 0x39896$	4021f4
3	p3_case_2	$5x + 0x2444b$	4021c4
4	p3_case_6	$(x \oplus 0x3bae) + (x >> 1)$	402226～40222d
5	p3_case_7	$5x + 0x9afe$	402234
6	p3_case_3	$(x \oplus 0xba59) + (x >> 2)$	4021d6～4021de
7	p3_case_5	$(4x) \oplus 0xb530$	402204、40220b

5.2 从分支指令恢复计算关系

```
# 倍增、加法与逻辑右移的代表分支
4021a4: lea 0xd61d(%rdi,%rdi,2),%eax
4021b4: lea (%rdi,%rdi,1),%eax
4021b7: xor $0xacac,%eax
4021f4: lea 0x39896(%rdi,%rdi,4),%eax
4021c4: lea 0x2444b(%rdi,%rdi,4),%eax
402224: mov %edi,%eax
402226: shr %edi
402228: xor $0x3bae,%eax
40222d: add %edi,%eax
402234: lea 0x9afe(%rdi,%rdi,4),%eax
4021d4: mov %edi,%eax
4021d6: shr $0x2,%edi
4021d9: xor $0xba59,%eax
4021de: add %edi,%eax
402204: lea 0x0(,%rdi,4),%eax
40220b: xor $0xb530,%eax
```

lea (%rdi,%rdi,4),%eax 计算的是 5x，lea (%rdi,%rdi,2),%eax 计算的是 3x。比较容易出错的是带右移的两个分支：它们一边保留原来的 x 做异或，一边对 x 右移，最后再相加，而不是把异或后的结果右移。

因此，我在 solve_phase_3 中按 selector 写出 switch，并分别计算八种情况。输出的第一项是 selector，第二项是选中分支算出的值。

5.3 运行验证

图 3 显示本关公开测试 10/10。seed 20261030 的 challenge 为 0x60db877a835f9c30，其最低三位为 0，因此该次运行进入 p3_case_1，最终显示“Phase 3 defused.”。

```
python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
Phase 3 challenge: 0x60db877a835f9c30
Input (unsigned decimal integers):
Phase 3 defused.
```

图 3　Phase 3 公开测试与额外 seed 验证

6 Phase 4 递归与返回值加工

6.1 区间划分与分支方向

phase_04 由 challenge 的最低七位得到 target，初始区间为 [0,127]，key 由高 32 位与 C>>7 的低 32 位异或。对应的入口设置为 edi=target、esi=0、edx=127、ecx=key，再调用 p4_walk。

$$t = U32(C)\,\&\,127;\quad k = H \oplus U32(C >> 7);\quad y = F(t, 0, 127, k)$$

```
# phase_04 调用参数
4023e1: xor %esi,%esi
4023e3: shr $0x20,%rax
4023e7: shr $0x7,%rcx
4023eb: and $0x7f,%edi
4023ee: mov $0x7f,%edx
4023f3: xor %eax,%ecx
4023f5: call 402320 <p4_walk>
```

```
# p4_walk 中点与判断
402324: mov %edi,%eax
402326: mov %edx,%edi
402328: sub %esi,%edi
40232a: shr %edi
40232c: add %esi,%edi
40232e: cmp %eax,%edi
402330: je 402380 <p4_walk+0x60>
402337: lea (%rdi,%rcx,1),%ebx
40233e: cmp %edi,%eax
402340: jb 402360 <p4_walk+0x40>
```

函数进入时，edi 保存 target，先复制到 eax；之后 edi 又被用于计算 high−low，并进一步得到 mid=low+((high−low)>>1)。等于 mid 时直接结束；若 target<mid 就走左分支，否则走右分支。单看这部分，确实很像平时写的二分查找。

6.2 递归返回阶段的计算

```
# 右递归与回溯
402342: lea 0x1(%rdi),%esi
402345: mov %eax,%edi
402347: call 402320 <p4_walk>
40234c: lea (%rax,%rax,2),%eax
40234f: xor %ebx,%eax
402356: xor $0x7,%eax

# 左递归与回溯
402360: lea -0x1(%rdi),%edx
402363: mov %eax,%edi
402365: call 402320 <p4_walk>
40236a: lea (%rax,%rax,2),%eax
40236d: xor %ebx,%eax
402374: xor $0x1d,%eax

# 递归终止
402380: lea 0x78(%rdi),%eax
402383: xor %ecx,%eax
```

$$m = l + ((h - l) >> 1)$$

$$F(t,l,h,k) = (m + 0x78) \oplus k, \quad t = m$$

$$F(t,l,h,k) = (3F(t,l,m-1,k) \oplus (m+k)) \oplus 0x1d, \quad t < m$$

$$F(t,l,h,k) = (3F(t,m+1,h,k) \oplus (m+k)) \oplus 7, \quad t > m$$

难点出现在递归返回以后。命中中点时只是算出了最内层的一个初值；返回上一层时，还要将子递归的结果乘 3，再与 mid+key 和对应方向的常数异或。左分支使用 0x1d（29），右分支使用 7。因此，不能只还原搜索路径，还要把沿途各层的返回运算一起还原。

6.3 寄存器生命周期与实现对应

表 4　p4_walk 中同一寄存器的不同语义

阶段	eax / edi	rbx 的作用
入口与中点	eax 暂存 target；edi 从 target 变为 mid。	尚未形成本层待保留值。
递归调用前	mov %eax,%edi 恢复 target，作为子调用第一个参数。	ebx=mid+key；rbx 属于被调用者保存寄存器。
call 返回后	eax 已是子递归返回值 r；lea 得到 3r。	本层 mid+key 仍可用于异或；结束前恢复调用者原 rbx。

最初我容易把 eax 一直当作 target。其实一旦执行 call 并返回，eax 中装的就是子函数结果；此后的 lea 是在计算该结果的 3 倍。与此同时，ebx 保存当前层的 mid+key，供递归返回后使用。后来把这两个值分清，整个递归式才比较顺畅地写出来。

```
uint32_t mid=low+((high-low)>>1);
if(target==mid) return (mid+0x78)^key;
/* 左分支的返回加工 */
return ((3*r)^(mid+key))^29;
/* 右分支的返回加工 */
return ((3*r)^(mid+key))^7;
```

6.4 运行验证

图 4 显示公开测试 10/10。额外 seed 20261030 对应 target=90，按照中点计算依次经过 63、95、79、87、91、89、90，最终显示“Phase 4 defused.”。这组中点由 challenge 和递归规则计算得到。

```
python3 public_test.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
Phase 4 challenge: 0xd1e0cdee6a36cdda
Input (unsigned decimal integers):
Phase 4 defused.
```

图 4　Phase 4 公开测试与额外 seed 验证

7 Phase 5 结构体内存与链表遍历

7.1 节点字段与逻辑后继

p5_traverse 通过头指针表找到起始节点。读取(%rdx)得到 value，0x4(%rdx)得到 extra，读取 0x8(%rdx)则取得下一个节点的指针。由此可以确定每个节点占 16 字节：两个 4 字节整数，后面跟一个 8 字节指针。

表 5 炸弹节点的内存布局

字节偏移	字段	宽度与含义
0～3	value	uint32_t，参与异或
4～7	extra	uint32_t，参与加法并作为末节点输出
8～15	next	64 位指针，保存逻辑后继地址

mov 0x8(%rdx),%rdx 是我读这关时最容易绕进去的一句。它先找到当前节点偏移 8 字节的 next 字段，读取里面保存的地址，再用该地址更新 rdx，并不是简单地执行 rdx+=8。

表 6 本人代码记录的十二个节点

节点编号	value	extra	next 编号
0	0x30d8	0x0929	7
1	0x3ff4	0x0b38	4
2	0x459b	0x0336	10
3	0xa9ef	0x081c	11
4	0x9e20	0x0769	0
5	0x1be9	0x0467	NULL
6	0x6366	0x006b	5
7	0x2a98	0x0234	2
8	0xd198	0x0131	9
9	0x6a54	0x0650	1
10	0xa5ff	0x0a8a	6
11	0x69cd	0x0550	8

节点按照连续地址存放，每个相隔 0x10 字节；但实际遍历要看 next 字段，而不是物理位置。表 6 按内存顺序给节点编号，我在 C 代码里则用数组下标表示 next，用−1 表示 NULL。这里的数组只是为了方便模拟，不能把它当作炸弹中原封不动的结构体定义。

$$3 \to 11 \to 8 \to 9 \to 1 \to 4 \to 0 \to 7 \to 2 \to 10 \to 6 \to 5 \to NULL$$

根据 next 逐个连接，得到 3→11→8→9→1→4→0→7→2→10→6→5→NULL。比如节点 3 的下一项是 11，节点 4 的下一项又是 0，这与按地址递增完全不同。八个起始头指针分别对应这条链的前八个位置。

7.2 初始节点为何在循环之外

```
# p5_traverse 选择与首次处理
40241d: and $0x7,%eax
402420: shr $0x3,%rcx
402424: mov 0x405b60(,%rax,8),%rdx
40242f: shr $0x20,%rdi
402433: shr $0x6,%rax
402437: mov 0x4(%rdx),%esi
40243a: xor %edi,%eax
40243c: xor (%rdx),%eax
40243e: mov %ecx,%edi
402440: xor $0x3711,%eax
402445: mov %esi,(%r8)
402448: add %esi,%eax
40244a: and $0x3,%edi
40244d: je 402469 <p5_traverse+0x59>
```

```
# 继续沿 next 移动
40244f: mov $0x1,%ecx
402454: mov 0x8(%rdx),%rdx
402458: add $0x1,%ecx
40245b: mov 0x4(%rdx),%esi
40245e: xor (%rdx),%eax
402460: add %esi,%eax
402462: mov %esi,(%r8)
402465: cmp %ecx,%edi
402467: jae 402454 <p5_traverse+0x44>
```

$$j = C\ \&\ 7;\quad s = (C >> 3)\ \&\ 3;\quad r = U32(C >> 6) \oplus H;\quad n_0 = heads[j]$$

$$r_0 = (r \oplus value(n_0) \oplus 0x3711) + extra(n_0)$$

$$n_i = next(n_{i-1});\quad r_i = (r_{i-1} \oplus value(n_i)) + extra(n_i),\quad 1 \le i \le s$$

and $3,%edi 是在起始节点已经参与计算之后才出现的。因此 steps=0 也会处理一个节点；steps>0 时，每轮先跟随 next，再计算当前节点的贡献。总处理节点数是 steps+1。常数 0x3711 只在第一次处理节点时使用。

函数把累计结果放在 eax 中返回，同时通过指针参数写出最后一个节点的 extra。于是这一关要输入两个整数：遍历计算出的累计值，以及末节点的 extra。

```
result=result^(node->value);
result=result^0x3711;
result+=node->extra;
for(uint32_t i=0;i<steps;i++) {
    cur=node->next;
    node=&p5_nodes[cur];
    result=result^(node->value);
    result+=node->extra;
    last_extra=node->extra;
}
```

从八个头中最靠后的节点 7 出发，再最多移动 3 次，也只到链中的第 11 个节点 6，尚未越过尾端；这与解法在循环内不另作 NULL 分支相一致。

7.3 运行验证

图 5 显示公开测试 10/10。seed 20261030 时，起始编号 j=2，steps=3，节点依次为 8→9→1→4，最后一个节点的 extra 是 0x0769，运行结果为“Phase 5 defused.”。

```
python3 public_test.py --phase 5
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
Phase 5 challenge: 0xa295dc5687270f3a
Input (unsigned decimal integers):
Phase 5 defused.
```

图 5　Phase 5 公开测试与额外 seed 验证

8 Phase 6 函数指针与两次间接调用

8.1 分派入口中的完整数据流

phase_06 本身只负责读取一个整数并调用 p6_dispatch。真正需要追踪的是 p6_dispatch 内部两次通过函数指针表进行的间接调用。两次使用的索引分别来自 challenge 的不同三位字段，可能恰好选中同一种运算。

```
# p6_dispatch
40255c: shr $0x6,%rdi
402563: shr $0x20,%rax
402567: xor %eax,%edi
402570: and $0x38,%ebx
402573: and $0x7,%eax
402576: call *0x405e00(,%rax,8)
40257d: xor $0x5624,%eax
402582: mov %eax,%edi
402584: call *0x405e00(%rbx)
40258f: add $0x1acb,%eax
```

$$x = U32(C >> 6) \oplus H;\quad i = C \,\&\, 7;\quad j = (C >> 3) \,\&\, 7$$

$$r_1 = T_i(x);\quad r_2 = T_j(r_1 \oplus 0x5624);\quad y = r_2 + 0x1acb$$

第一次调用用 eax 中的索引乘 8 查表。第二次调用没有写比例因子，是因为 rbx 里已经保存了 challenge&0x38，本身就是 8 的倍数，可以直接作为字节偏移。两次调用之间还要将第一次返回值与 0x5624 异或，因此第二次不是对原始 x 重新计算。

数据流可以写成：challenge → x → T_i → 异或 0x5624 → T_j → 加 0x1acb。第一次的结果必须参与第二次输入，不能将两次调用独立计算后相加。

表 7　八项函数指针表及六种运算

索引	对应函数	T 索引(x) 的 32 位关系
0	p6_op_3	ROR32(x,5)
1	p6_op_2	((x >> 1) ⊕ x) + 0xdab5
2	p6_op_4	ROL32(x,3) ⊕ 0x0981
3	p6_op_5	(x ⊕ 0x9c42) + 2x
4	p6_op_0	(x + 0xf4b2) ⊕ (x >> 2)
5	p6_op_2	与索引 1 相同
6	p6_op_1	(32x + 0xaad7) ⊕ (7x)
7	p6_op_2	与索引 1 相同

8.2 算术运算与循环移位

```
# 六种实际操作的关键指令
4024e4: lea 0xf4b2(%rdi),%eax
4024ea: shr $0x2,%edi
4024ed: xor %edi,%eax
4024f6: lea 0x0(,%rdi,8),%edx
4024fd: shl $0x5,%eax
402500: sub %edi,%edx
402502: add $0xaad7,%eax
402507: xor %edx,%eax
402516: shr %eax
402518: xor %edi,%eax
40251a: add $0xdab5,%eax
402526: ror $0x5,%eax
402536: rol $0x3,%eax
402539: xor $0x981,%eax
402546: xor $0x9c42,%eax
40254b: lea (%rax,%rdi,2),%eax
```

$$ROL32(x,n) = U32((x << n) \mid (x >> (32-n)))$$

$$ROR32(x,n) = U32((x >> n) \mid (x << (32-n)))$$

rol、ror 与普通移位的区别是移出去的比特会从另一侧回卷。这里循环左移 3 位、循环右移 5 位，都能直接按 32 位数实现。p6_op_1 还把 8x−x 得到 7x，并在另一条运算路径里得到 32x，最后异或两条结果。

第三关的跳转表用 jmp 转入某个分支；第六关的函数指针表则用 call 执行函数，随后回到分派器继续计算。虽然都需要从表里取地址，但控制流不同。对应的函数与运算整理在表 7。

```
uint32_t r1=p6_apply(index1,x);
uint32_t r2=p6_apply(index2,r1^0x5624);
values[0]=r2+0x1acb;
```

8.3 运行验证

图 6 显示本关公开测试 10/10。seed 20261030 时，索引分别为 i=3、j=1，先调用 p6_op_5，再调用 p6_op_2，最终显示“Phase 6 defused.”。

```
python3 public_test.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
Phase 6 challenge: 0x8156aef15aa39d4b
Input (unsigned decimal integers):
Phase 6 defused.
```

图 6 Phase 6 公开测试与额外 seed 验证

9 Honor：字符串布局、上下文与校验

9.1 长度与栈上复制范围

Honor 的接口多了 context 这一参数。检查 defuser_main.c 可以看到，程序会先计算前六关输出，再交给 transcript_context 生成上下文，并把它与第七关 challenge 一同传给 solve_secret。因此，Honor 不能只看最后一关的 challenge。

```
# secret_gate 长度与相邻位置检查
40271b: lea -0xd0(%rbp),%r12
402730: rep stos %rax,%es:(%rdi)
402733: movl $0x0,(%rdi)
40273c: call 401190 <_init+0x190>
402741: sub $0x24,%rax
402745: cmp $0x7f,%rax
402749: jbe 402760 <secret_gate+0x60>
402760: mov %rbx,%rsi
402763: mov %r12,%rdi
402766: call 4026e0 <secret_copy>
40276b: cmpl $0x4e45504f,-0xb0(%rbp)
40277d: lea -0xac(%rbp),%rdi
402784: call 402620 <secret_phase>
```

```
# secret_copy 逐字节复制
4026e4: xor %eax,%eax
4026f0: movzbl (%rsi,%rax,1),%edx
4026f4: mov %dl,(%rdi,%rax,1)
4026f7: add $0x1,%rax
4026fb: test %dl,%dl
4026fd: jne 4026f0 <secret_copy+0x10>
```

长度检查先计算 len−0x24，再通过无符号比较限制它不大于 0x7f，因此允许的字符串长度是 36～163 字节。小于 36 的长度在无符号减法后不会通过检查。secret_copy 则逐字节复制输入，连末尾的 NUL 也会一起写入。

复制起点为 rbp−0xd0。rep stos 的 20 个八字节写入加随后四字节清零，共清理 164 字节；结合长度上限，最大字符串 163 字节加 NUL 恰好落在这一范围内。这里据指令描述有效写入区域，不把它擅自还原成某个未经源码确认的局部结构体声明。

9.2 OPEN 与后续整数的位置

表 8 Honor 输入字符串的字节布局（偏移从 0 开始）

输入偏移	栈中对应位置	内容与检查
0～31	rbp−0xd0 ～ rbp−0xb1	32 字节前缀；本人使用 32 个 ASCII 字符 A。
32～35	rbp−0xb0 ～ rbp−0xad	固定 ASCII 字节 OPEN，与 0x4e45504f 比较。
36 起	rbp−0xac 起	解析 a、b、c 三个无符号十进制整数；数字间以空格分隔。
len	复制区内的末尾	NUL 终止符；len 为 36～163，格式校验仍须成功。

检查地址与复制起点的差为 (−0xb0)−(−0xd0)=0x20，即 32；整数解析起点之差为 0x24，即 36。在 x86-64 小端序中，0x4e45504f 从低地址到高地址的字节依次是 4f、50、45、4e，对应 O、P、E、N。故 OPEN 位于偏移 32～35，也就是从 1 开始计数的第 33～36 个字节。

这一关比较容易忽略的是字符串的位置要求。程序会检查偏移 32～35 处是否为 OPEN，然后从偏移 36 处开始读取三个整数。因此，我在构造答案时先输出 32 个固定字符，再接上 OPEN 和计算得到的三个整数。如果前缀多一个或少一个字符，OPEN 就无法出现在指定位置，即使后面的三个数都算对了，也无法通过检查。

```
"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN%" PRIu32 " %" PRIu32 " %" PRIu32
```

9.3 challenge 与 context 的数值关系

```
# secret_phase 第一项与折叠
402680: mov %r13d,%ecx
402685: and $0x3,%ecx
402688: cmp %ecx,-0x2c(%rbp)
402690: imul $0x4e64,%ecx,%r9d
40269a: shr $0x20,%rsi
40269e: shr $0x20,%r8
4026a2: xor %r12d,%esi
4026a5: xor %r13d,%r8d
4026a8: shr $0x10,%r13
4026ae: xor %r13d,%r9d
4026b1: xor %r8d,%edi
4026b4: call 402600 <secret_mix>
4026b9: xor %r9d,%eax
4026bc: cmp %eax,-0x28(%rbp)

# 第三项与 secret_mix
4026c1: shr $0x7,%esi
4026c4: lea 0x4595(%r8,%rcx,1),%edx
4026cc: add %esi,%edx
4026ce: xor %edx,%eax
4026d0: cmp -0x24(%rbp),%eax
402606: rol $0x7,%eax
402609: add $0x4e64,%eax
40260e: xor $0x4595,%eax
```

$$A = U32(C >> 32) \oplus U32(C);\quad B = U32(K >> 32) \oplus U32(K)$$

$$a = U32(K)\,\&\,3;\quad u = A \oplus B;\quad v = (0x4e64 \times a) \oplus U32(K >> 16)$$

$$mix(u) = (ROL32(u,7) + 0x4e64) \oplus 0x4595;\quad b = mix(u) \oplus v$$

$$c = b \oplus (B + a + 0x4595 + (A >> 7))$$

第一项 a 取 context 的最低两位。challenge 和 context 各自的高、低 32 位分别异或，得到 A、B，再组合成 u；另外由 a 和 context 右移后的结果得到 v。secret_mix(u)与 v 异或得到第二项 b，第三项 c 则继续由 b 及 A、B 计算。

这里有两处不同的右移容易混淆：context>>16 先对 64 位数据移位，之后才截成 32 位；而 A>>7 已经是在 32 位无符号数上右移。两种写法不能互换。

```
uint32_t u=A^B;
uint32_t v=(0x4e64*a)^(uint32_t)(context>>16);
uint32_t b=secret_mix(u)^v;
uint32_t c=b^(B+a+0x4595+(A>>7));
```

输出时使用 snprintf 拼接固定前缀、OPEN 和三个整数，并检查缓冲区是否足够。这样既满足固定偏移，也能保证三个数按照无符号十进制格式输出。

9.4 实际运行结果

图 7 为 honor_public.png 的测试输出局部，十个公开 seed 全部 PASS，显示 Secret public tests: 10/10。完整运行的额外 seed 命令与 Secret phase defused. 结果见第 10 节图 9。

```
yeziming@yzm:~/ICS2026/ics-2026-bomblab$ make
python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

图 7　Honor 公开测试 10/10

10 测试记录与结果汇总

10.1 公开测试与证据对应

课程公开测试使用的十个 seed 为 0、1、7、42、99、123、2026、65535、987654321、18446744073709551615。图 1～6 分别展示各关 10/10，图 7 展示 Honor 的 10/10，图 8 则给出 make test 的汇总结果。

表 9　各关实际验证记录

关卡	公开测试	非公开列表 seed	截图证据
Phase 1	10/10	20261030，通过	图 1
Phase 2	10/10	20261030，通过	图 2
Phase 3	10/10	20261030，通过	图 3
Phase 4	10/10	20261030，通过	图 4
Phase 5	10/10	20261030，通过	图 5
Phase 6	10/10	20261030，通过	图 6
Honor	10/10	20261030，完整运行通过	图 7、图 9

```
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

图 8　全部公开测试汇总

10.2 额外 seed 的完整验证

seed 20261030 不在公开测试列表中。除了分别检验六个正式关卡，我还使用这一 seed 将六关输出与 Honor 输入连续送入 bomb，完整运行的命令如下：

```
(./defuser --seed 20261030;
 ./defuser --secret --seed 20261030) | ./bomb --seed 20261030
```

图 9 显示六关均已解除，并出现“Congratulations! You've defused the bomb.”和“Secret phase defused.”。



图 9 seed 20261030 完整拆弹与 Honor 验证

这些结果证明已测试的输入能够通过个人炸弹；是否覆盖全部隐藏输入，还要依靠各关计算关系本身的正确性。

11 遇到的问题与解决

11.1 Phase 4 二分查找带来的惯性思维

一看到 p4_walk 中的中点计算和左右递归，我还挺有信心：二分查找以前做过很多次，应该不难。最初我一直顺着区间怎么缩小往下看，甚至觉得只要搞清楚 target 和 mid 的关系就差不多了。可是写到返回值时才发现完全不是这么回事：找到目标只是递归的终点，真正要输出的数字还在每一层返回时被重新计算。

我后来重新从 call 后面的指令读起，才注意到 eax 已经不再是原来的 target，而是下层递归算出来的值。再把 rbx 保存的 mid+key 一起放进式子里，左右两条分支就能接上了。这个小插曲给我的印象很深：熟悉的算法框架确实能帮助判断方向，但也可能因此跳过一些重要的语句，在歧途上越走越远。

11.2 Phase 5 从连续地址到逻辑链表

第五关让我卡住的时间比预想中长。GDB 里一串节点挨着排放，按地址整理成表格并不难，可是真正顺着代码走时，我总会不自觉地把“内存里的下一个节点”和“链表里的下一个节点”当成一回事。尤其看到 0x8(%rdx)这种写法，脑子里很容易先想到地址加 8。

后来我把每个节点的 next 字段单独整理出来，才发现编号 3 的后继是 11，而编号 4 的后继是 0，链表的实际连接顺序与节点在内存中的排列顺序完全不同。再回头看 mov 0x8(%rdx),%rdx，就容易理解了：0x8(%rdx)表示当前节点中 next 字段所在的位置，mov 则将该字段保存的后继地址读入 rdx，供下一次遍历使用。之前一直把节点地址和指针指向的地址混在一起，在这里绕了非常之久。

11.3 Phase 6 ——“意料之外”的“短”代码

做到第六关，看到 phase_06 只有不长的一段汇编，我确实松了一口气，想着前面已经分析了相对复杂的递归和链表，这一关总该轻松些了。结果顺着 call 进入 p6_dispatch 才发现，后面还有一张八项函数指针表，拆开又是好几个运算函数；更麻烦的是，它居然还要连续调用两次！

起初我只能一边看寄存器一边往回翻函数，特别容易把两次调用的参数混在一起。后来干脆只画一条数据流：challenge 取出索引和初值，第一次函数给出 r1，异或后再进入第二个函数，最后加常数。我把两次函数调用的输入和返回值分别记下来，再回头对应汇编中的寄存器变化，整个计算过程就清楚多了。

11.4 Honor 先理解字符串如何进入比较

Honor 开始时，我还习惯像前六关那样寻找最后要输入的几个整数，结果遇到 secret_gate 和 secret_copy 就有点摸不着头绪：为什么要先检查整行长度？明明是在算数字，为什么还要复制字符串、比较一段固定的十六进制常数？

等我把复制起点和检查地址的距离算出来，才发现 OPEN 必须卡在第 32 字节的位置；小端序又解释了那个看起来很奇怪的常数。把字符串的布局先画出来，再往后看三个整数的计算，一下子就顺畅多了。回头想想，这一关最容易忽略的不是某个运算符，而是输入在参与比较之前究竟被放到了哪里。

12 实验体会与思考

12.1 从指令含义走向程序行为

这次实验真正做下来，我才感觉到“认识汇编指令”和“看懂一个程序”之间差得还挺远。单看 mov、lea、xor，大多数时候我知道它们在做什么；但如果没有追清楚数据从哪个寄存器来、接下来又被谁修改，几条都认识的指令放在一起仍然会让我迷糊。第四关的递归、第五关的指针和第六关的间接调用，都给我好好上了一课。

后来我越来越习惯先把需要追踪的值写下来，再沿着调用和内存访问一点点对照。做完一关看到 PASS，感到非常开心，但更让我踏实的是，自己能够解释为什么它会 PASS，而不是只留下一个碰巧对上的数字。

12.2 通关之后的意外惊喜

说起来有些惭愧，做前六关的时候，我其实没有太关注剧情。那时满脑子都是寄存器、函数调用和各种计算关系，只想着尽快把汇编分析清楚，顺利拆完炸弹。直到最后完成 Honor，看到完整的故事呈现在眼前，我才终于放下那些让人头昏眼花的汇编，静下心来认真体会这座城市的故事。

回过头来看，故事里那些看似不起眼的细节其实很有意思。司机总说明天送货，维修铺的灯总说明天装好，可是因为每天都会恢复旧存档，这些“明天”始终没有到来。直到六关完成，城市的日期终于向前走了一天，那盏灯也真正亮了起来。而 Honor 里 sky 和许遥留下的故事，又让这份关于

时间和变化的思考多了一些现实的意味。两个人曾经一起做项目，后来各自走上不同的道路，但观测台上的那盏灯和两把椅子依然留在那里。“下次回来，先坐一会儿”，简单的一句话，却让人觉得很温暖。

更让我惊喜的是，助教们竟然在这样一个以汇编分析为主的实验里，认真设计了如此完整的剧情。整个过程让我回忆起之前参加过的旦旦城定：一路忙着解题，直到最后才发现，原来前面那些零散的线索一直在共同讲述一个故事。在连续分析了这么多关汇编之后，迎接我的不是又一道复杂的计算题，而是这样一个意料之外的彩蛋，确实让人惊喜。

尤其是读到“事已至此，先睡觉吧。”，再到最后的“谢谢你远道而来，与我相遇。晚安。”，那一刻真的有些触动。原本只是想完成一次课程实验，没想到最后会因为一段故事而停下来思考这么久。也许今天的努力不一定立刻带来什么改变，但只要明天还会到来，就总有机会继续尝试，走向新的方向。

很感谢助教老师为这次 Lab 花费的心思，让原本可能有些枯燥的汇编逆向多了一份特别的体验。也感谢在这门课程中与老师们的相遇。比起最后成功拆弹的提示，这段意料之外的故事，或许更是我完成实验后难以忘怀的部分。

13 参考资料与辅助工具

13.1 参考资料

[1] 复旦大学计算机系统基础，2026 秋季，Lab2 BombLab 实验说明。用于核对内容要求、Honor 规则、协作与学术诚信及提交格式。https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/ 。

[2] ICS-26Fall-FDU/BombLab，README.md、AGENTS.md。用于核对报告必交项目、允许的辅助范围与测试规则。https://github.com/ICS-26Fall-FDU/BombLab 。

[3] 本人 BombLab 仓库，defuser.c、defuser.h、defuser_main.c、public_test.py 及公开测试列表。用于核对个人代码、调用接口和测试数据。https://github.com/yzm7678024-glitch/ics-2026-bomblab

[4] System V AMD64 psABI，Register Usage。用于复核参数寄存器、返回寄存器与 rbx 的保存约定。https://gitlab.com/x86-psABIs/x86-64-ABI/-/blob/master/x86-64-ABI/low-level-sys-info.tex。

[5] GNU GDB 官方手册，Examining Memory。用于复核 x 命令的字节、字与八字节单位及内存观察的解释。https://sourceware.org/gdb/current/onlinedocs/gdb.html/Memory.html 。

[6] Bryant, R. E. 与 O’Hallaron, D. R. Computer Systems: A Programmer’s Perspective, 3rd ed., Chapter 3, Machine-Level Representation of Programs。课程背景：过程调用、控制流、数组和结构体；本报告具体常量以本人炸弹与代码为准。

[7] 本次实验原始材料：个人 bomb 及其反汇编文件 bomb_disassembly.txt、公开测试截图、seed 20261030 额外验证截图。用于核对指令、常量、数据表和实际运行结果。

13.2 工具用途与辅助说明

GCC 与 GNU Make 用于编译，objdump 用于反汇编，GDB 用于阅读运行中的寄存器和内存数据，课程测试脚本用于检查公开用例。

本次实验基于本人领取的 bomb 开展逆向分析，完成了 defuser.c 的实现与测试。实验过程中使用 Deepseek 辅助理解部分汇编指令、讨论程序逻辑，并借助 deepseek harness 整理了部分报告格式。

最终代码、关键计算关系和报告内容均由本人亲自完成，六关公开测试 60/60、Honor 公开测试 10/10 及额外 seed 验证结果均来自本人实际运行。